FSA-AstraInstall

Версия: V4.8.380 (2026-04-09 16:53)
Компания: ООО "НПА Вира-Реалтайм"
Разработчик: @FoksSegr & AI Assistant
Сайт: https://rlt.ru/

📖 О проекте

FSA-AstraInstall — профессиональная система автоматизации для управления Linux-системами с **графическим интерфейсом** и **консольным режимом**. Комплексное решение для автоматического обновления операционных систем, установки и управления компонентами Wine, настройки репозиториев и мониторинга процессов.

Экосистема FSA: этот репозиторий — разработка Linux-установщика; удалённая сборка бинарников выполняется через отдельный проект **FSA-DockerManager-Razrab**. Краткая схема репозиториев и ролей: `MEMORANDUM.md` в корне.

🎯 Ключевые возможности

- 🍷 **Управление Wine** — установка различных версий Wine, создание и управление WINEPREFIX, множественные экземпляры приложений
- ⚡ **Автоматизация системы** — полное обновление ОС с автоответами на интерактивные запросы
- 📦 **Менеджер компонентов** — установка/удаление Astra.IDE, Wine-приложений, winetricks библиотек
- 🔄 **Управление репозиториями** — проверка, активация/деактивация, автоматическая настройка APT-репозиториев
- 💿 **Офлайн-репозиторий** — универсальный архив с поддержкой нескольких ядер и инкрементальным дополнением
- 📊 **Мониторинг** — отслеживание процессов, использования ресурсов, изменений файловой системы
- 🎨 **Графический интерфейс** — современный tkinter-интерфейс с вкладками и визуализацией прогресса
- 💻 **Консольный режим** — полная автоматизация для серверов и удаленных систем

🏗️ Архитектура (63066 строк кода)

Модульная структура (после рефакторинга)

```
FSA-AstraInstall-Razrab/           # Репозиторий разработки (публичные
релизы — отдельное репо FSA-AstraInstall)
├── Code/                          # 📁 Модульная кодовая база
│   ├── Run/                       # 🚀 Точка входа (181 строка)
│   │   └── FSA-AstraInstall.py   # Главный исполняемый файл
```

```
│   │
│   ├── core/                      # ⚙️ Ядро системы (6,760 строк)
│   │   ├── config.py              # Конфигурация и константы
│   │   ├── process_runner.py      # Управление процессами
│   │   ├── progress.py            # Управление прогрессом
│   │   ├── activity.py            # Отслеживание активности
│   │   └── print_override.py      # Переопределение print()
│   │
│   ├── gui/                       # 🎨 Графический интерфейс (25,986
строк)
│   │   ├── application.py         # Главное окно приложения
│   │   ├── tabs/                  # Вкладки интерфейса
│   │   └── widgets/               # Виджеты и компоненты
│   │
│   ├── handlers/                  # 🔧 Обработчики (10,389 строк)
│   │   ├── wine_package.py        # Установка Wine
│   │   ├── wine_config.py         # Настройка Wine
│   │   ├── wine_environment.py    # Окружение Wine
│   │   ├── system.py              # Системные настройки
│   │   ├── winetricks.py          # Winetricks компоненты
│   │   ├── apt.py                 # APT пакеты
│   │   ├── application.py         # Wine приложения
│   │   └── shortcut.py            # Ярлыки рабочего стола
│   │
│   ├── managers/                  # 📋 Менеджеры (5,988 строк)
│   │   ├── component_status.py    # Статусы компонентов
│   │   ├── component_installer.py # Установка компонентов
│   │   ├── system_updater.py      # Обновление системы
│   │   └── wine_instance.py       # Экземпляры Wine
│   │
│   ├── runtime/                   # 🏃 Среда выполнения (2,056 строк)
│   │   ├── console_mode.py        # Консольный режим
│   │   ├── update_checker.py      # Проверка обновлений
│   │   ├── initialization.py      # Инициализация приложения
│   │   └── startup.py             # Запуск и процессы
│   │
│   ├── utils/                     # 🛠️ Утилиты (8,815 строк)
│   │   ├── path_utils.py          # Работа с путями
│   │   ├── text_utils.py          # Текстовые утилиты
│   │   ├── system_utils.py        # Системные утилиты
│   │   ├── platform_utils.py      # Определение платформы
│   │   └── ...                    # Другие утилиты
│   │
│   ├── monitoring/                # 📊 Мониторинг (1,348 строк)
│   │   ├── process_monitor.py     # Мониторинг процессов
│   │   ├── installation_monitor.py # Мониторинг установки
│   │   └── filesystem_monitor.py # Мониторинг файловой системы
│   │
│   ├── updater/                   # 🔄 Обновления (1,214 строк)
│   │   └── self_updater.py        # Самообновление приложения
│   │
│   └── interactive/               # 💬 Интерактивность (331 строка)
│       ├── config.py              # Конфигурация запросов
│       └── handler.py             # Обработка интерактивных запросов
```

```
│
├── RunScript/                      # Скрипты сборки, публикации, обновления
на SMB (GUI сборщика здесь нет)
│   ├── build.py                   # Тот же сценарий, что корневой build.py
(запуск из любой директории)
│   ├── build_stamp.py             # Проставление времени сборки в
Version.txt
│   ├── publish_build.py           # Публикация сборки
│   ├── publish_release.py         # Публикация релиза
│   ├── astra_update.py            # Копирование на SMB после сборки
(вызывается из build.py)
│   ├── commit_message.py          # Вспомогательный скрипт сообщений
коммита
│   └── fix-permissions.sh         # Права на дерево файлов в текущем
каталоге
│
├── Version.txt                     # Версия и время сборки (APP_VERSION,
BUILD_TIME)
├── build.json                      # Конфиг PyInstaller и вариантов сборки
(full / auto / light)
├── FSA-AstraInstall-1-7           # 📦 Пример имени бинарника (full,
astra-1.7); фактические имена — из build.json
├── FSA-AstraInstall-1-8           # 📦 Пример (full, astra-1.8); также
возможны суффиксы Auto / Min для других вариантов
├── MEMORANDUM.md                   # Связь с FSA-DockerManager-Razrab, FSA-
AstraWin-Razrab и публичными репо
├── README.md                       # 📄 Документация проекта
└── HELPME.md                       # 📖 Руководство пользователя
```

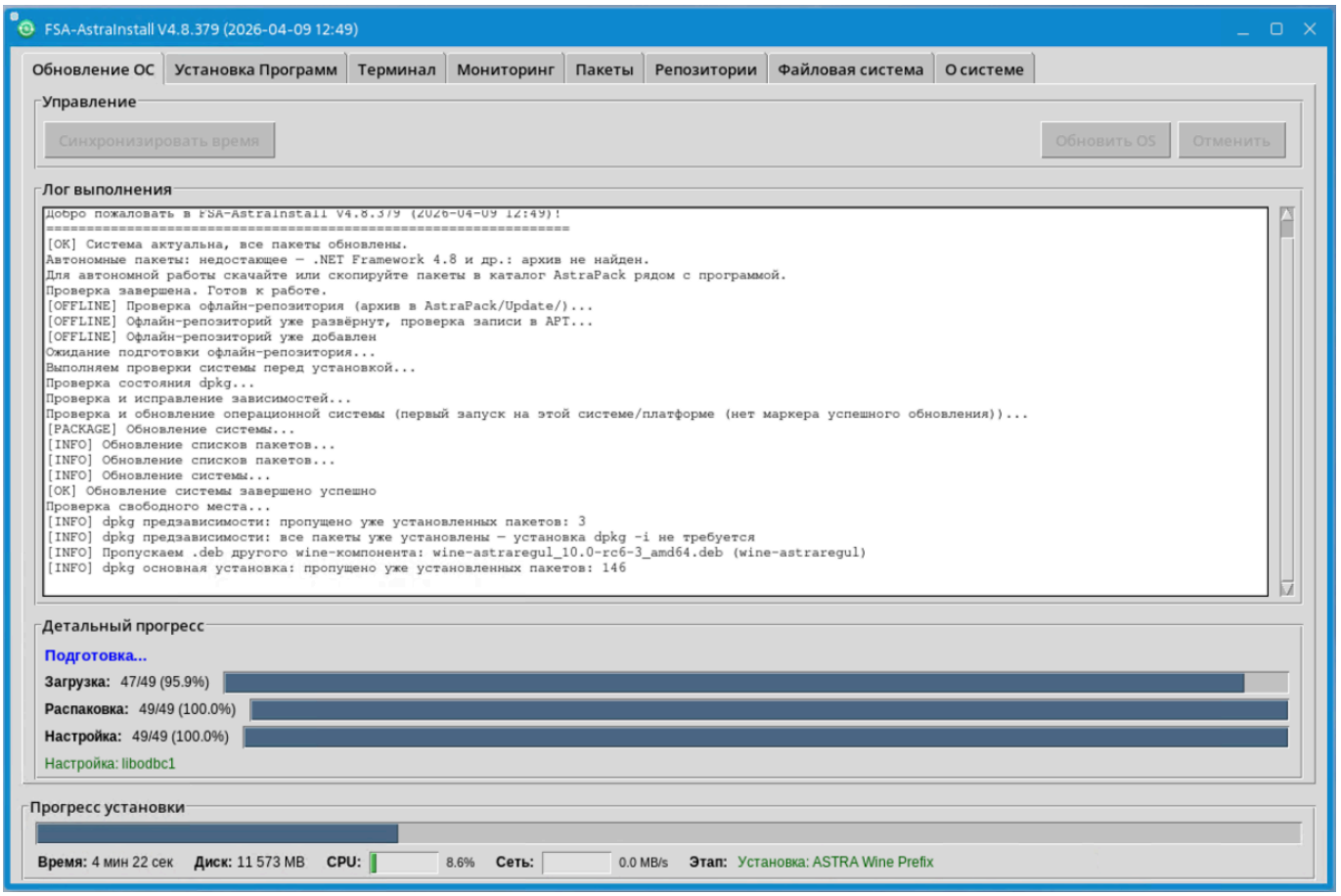
📊 Статистика проекта

Параметр	Значение
Архитектура	Модульная (десятки Python-модулей в `Code/`)
Точка входа	`Code/Run/FSA-AstraInstall.py`
Платформы	Astra 1.7, Astra 1.8, Ubuntu 20.04+, Debian 10+

Числа строк и размеры бинарников меняются от релиза к релизу; ориентир — актуальный `Version.txt` и артефакты после сборки.

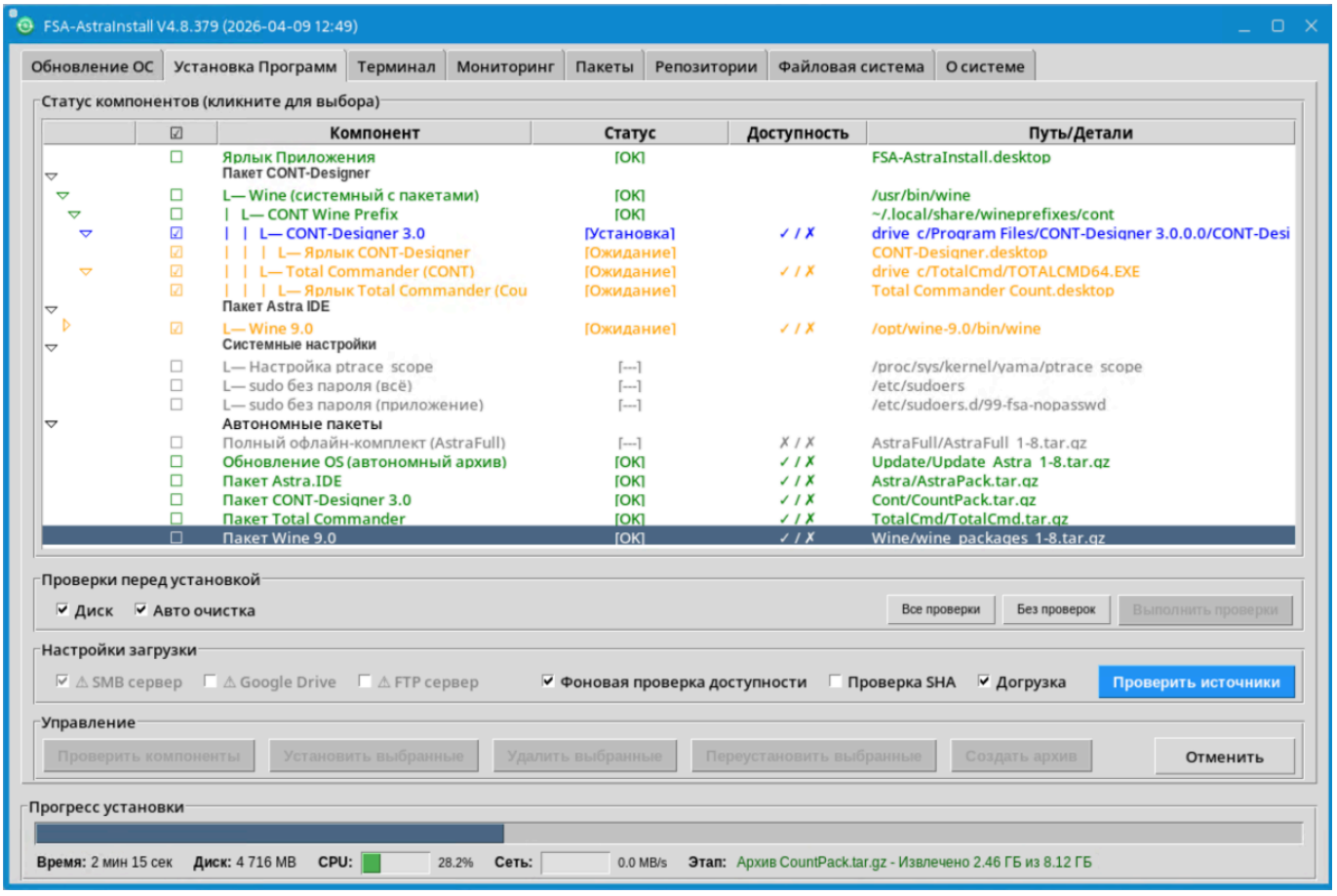
🖼️ Скриншоты интерфейса

Главная вкладка



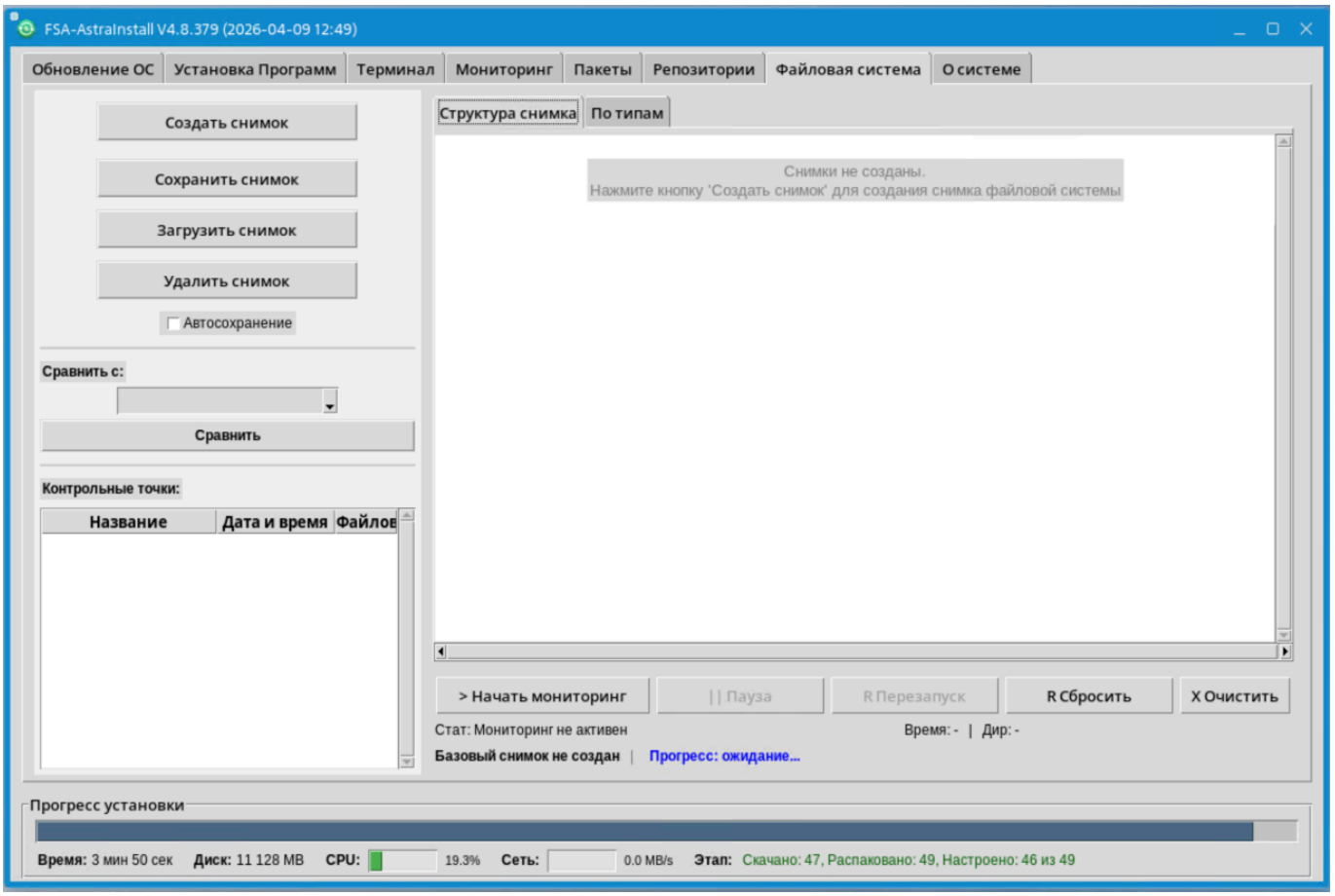
Обзор системы с информацией о компонентах и быстрым доступом к основным функциям

Вкладка Wine



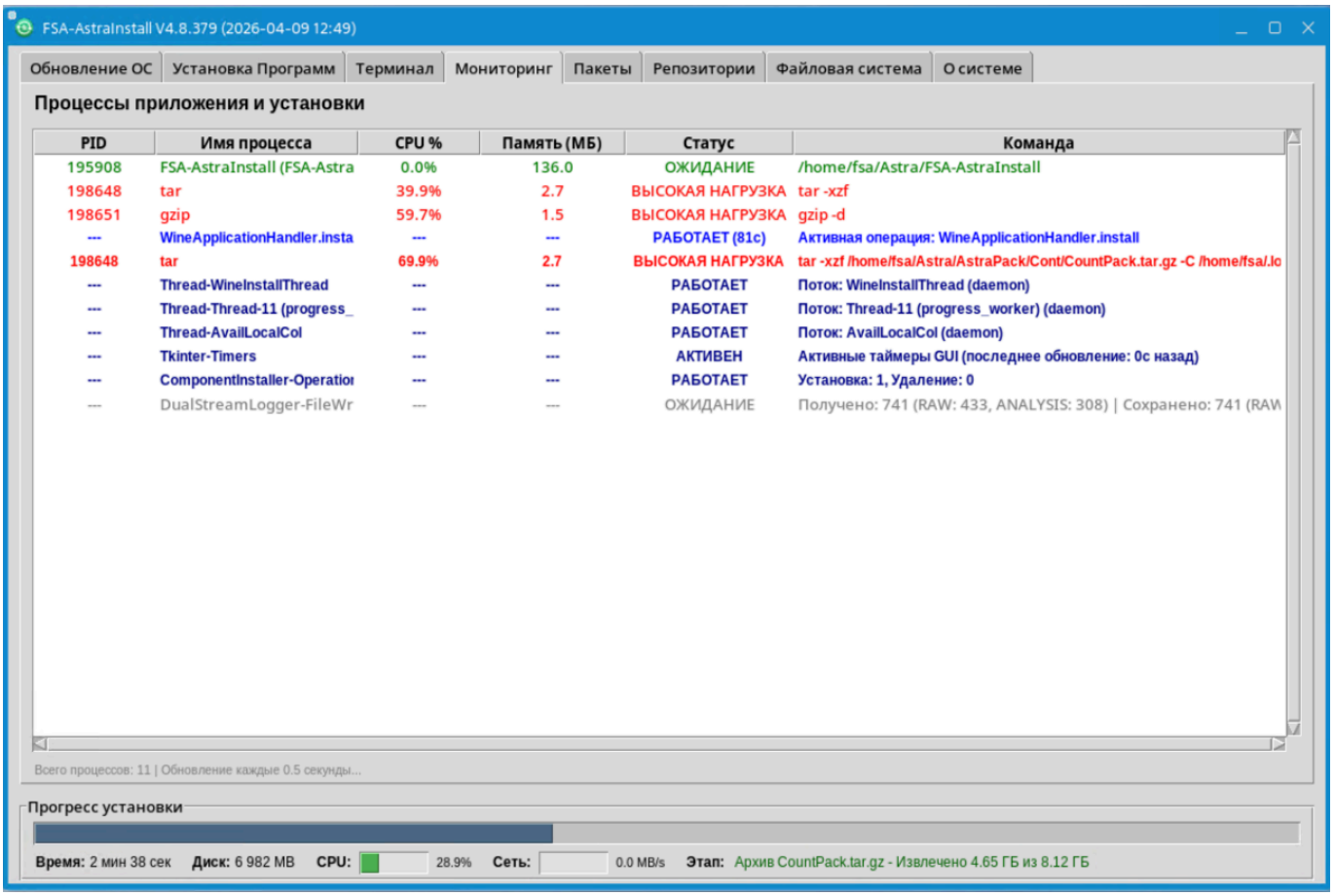
Управление Wine окружением: установка приложений, настройка WINEPREFIX, создание экземпляров

Мониторинг файловой системы



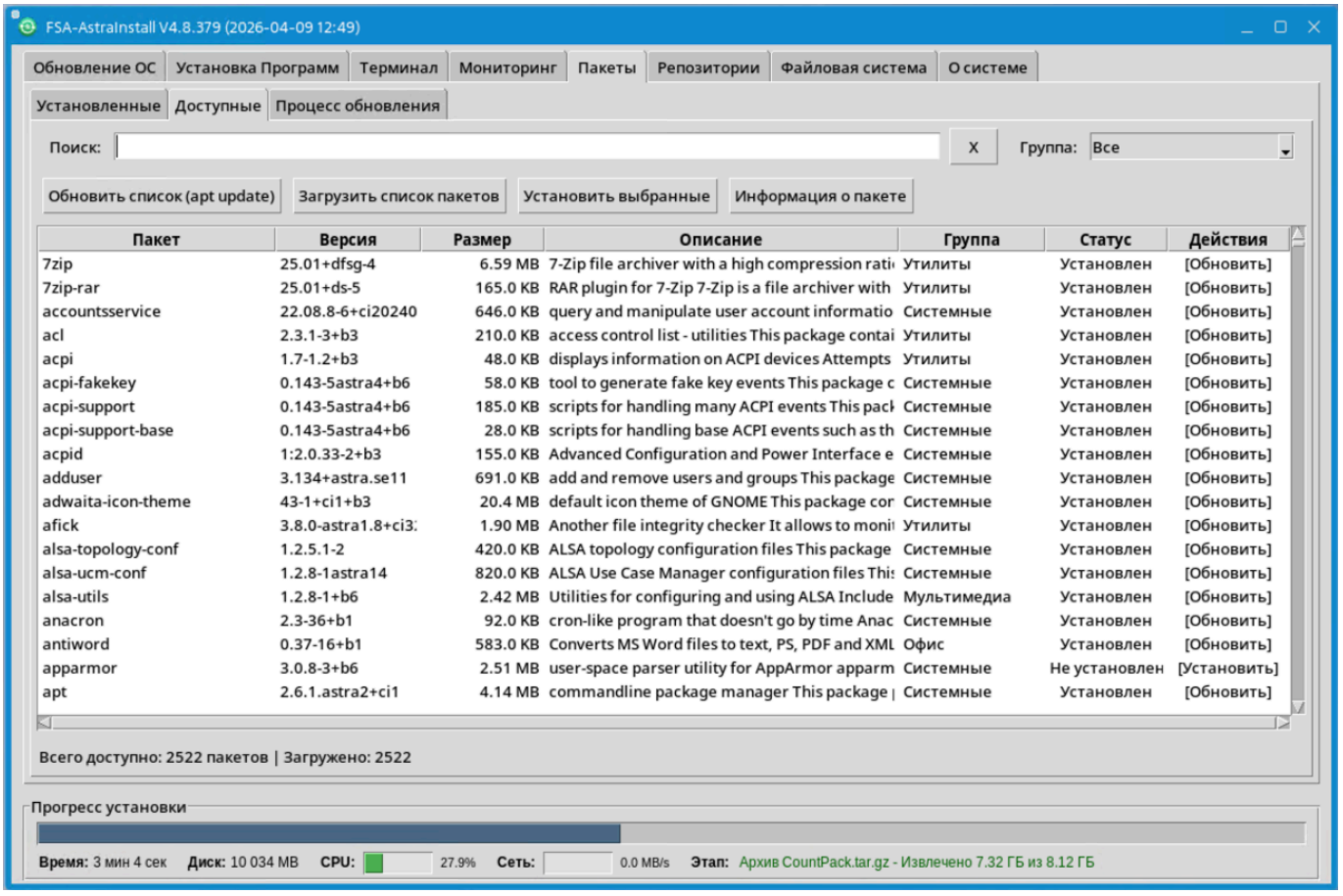
Отслеживание изменений файловой системы: создание снимков и сравнение до/после установки

Мониторинг процессов



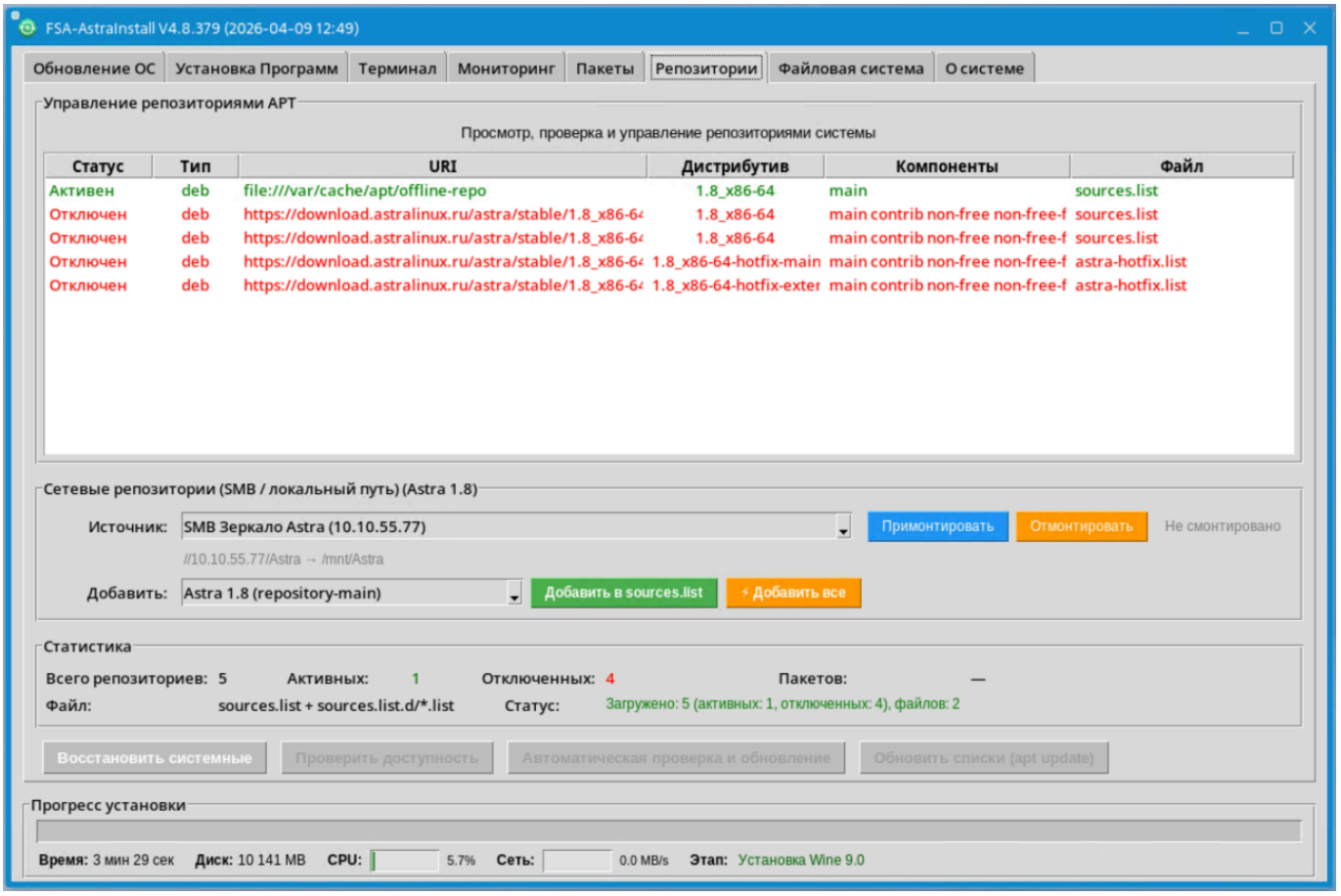
Отслеживание запущенных процессов с детальной информацией о потреблении ресурсов

Управление пакетами



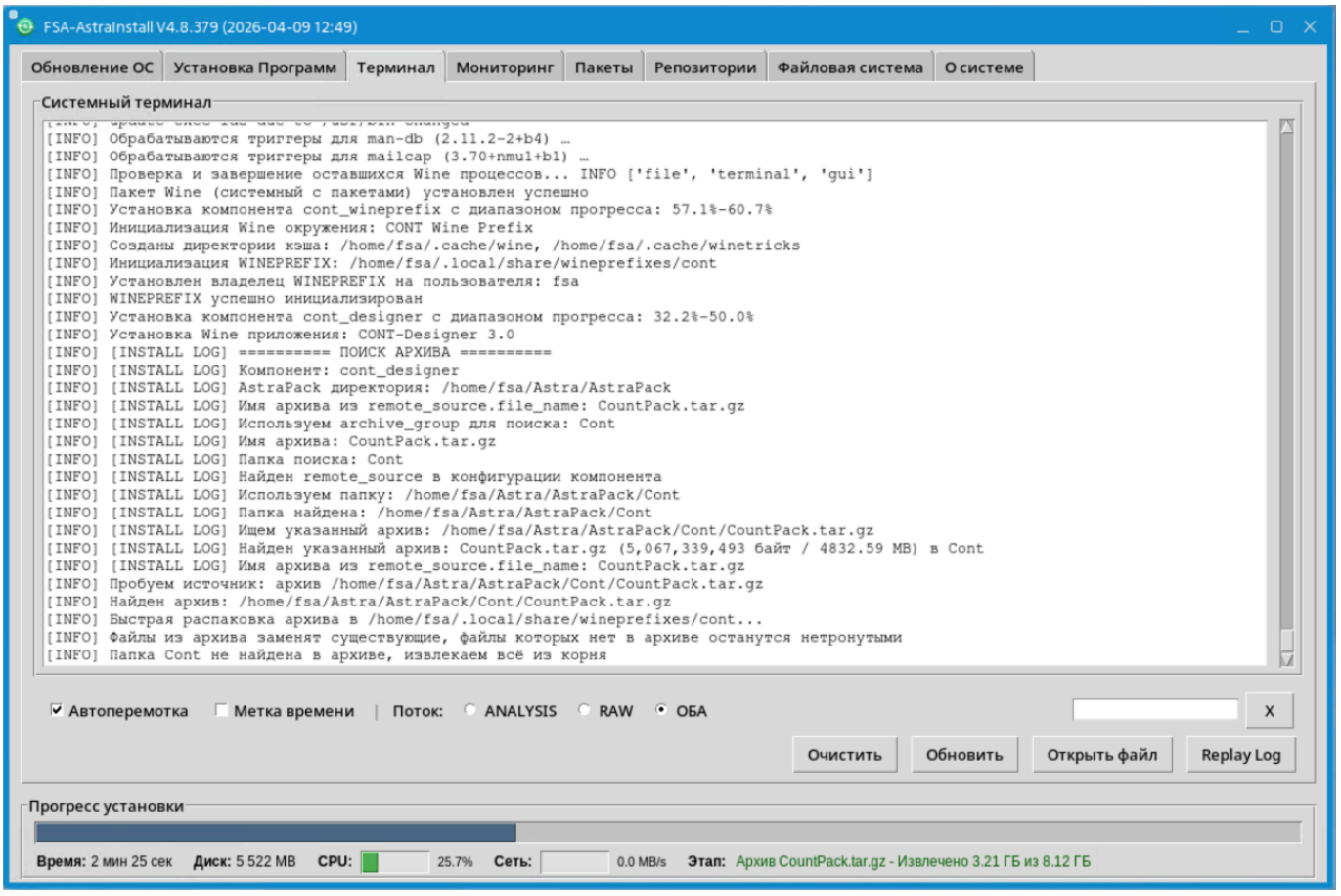
Установка и удаление APT пакетов через графический интерфейс

Настройка репозиториев



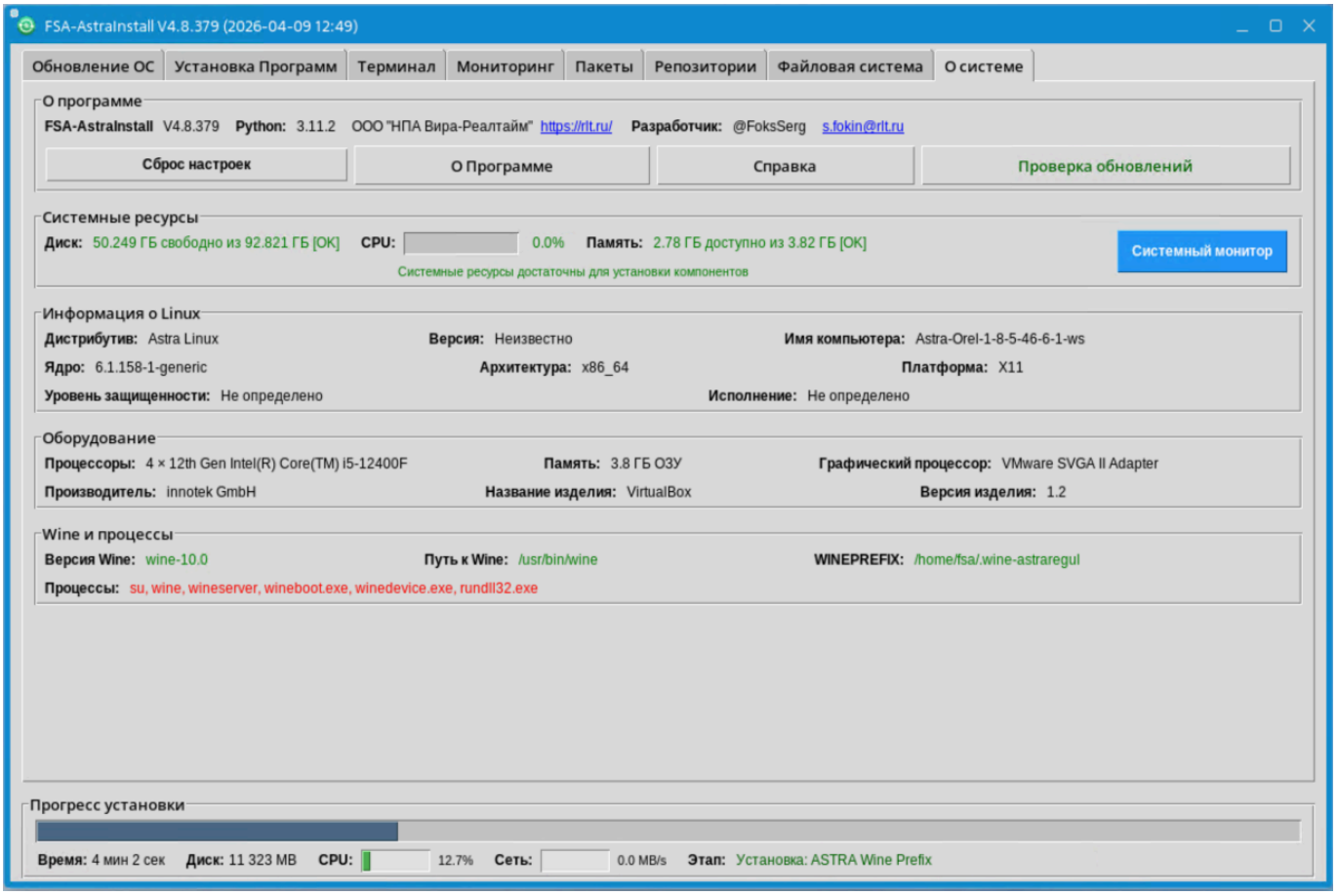
Проверка, активация и восстановление APT репозиториев

Терминал и логи



Просмотр логов выполнения команд в реальном времени

О системе



Информация о версии, документация и системный мониторинг

✨ Основные возможности

🍷 Управление Wine

Установка и настройка

- ✅ **Множественные Wine окружения** — поддержка различных WINEPREFIX с независимыми конфигурациями
- ✅ **Установка Wine версий** — wine_astraregul, wine_9 и другие сборки
- ✅ **Настройка Wine** — автоматическая конфигурация wine-gecko, winetricks компонентов
- ✅ **Управление экземплярами** — создание и управление множественными экземплярами Wine приложений

Работа с приложениями

- ✅ **Установка Windows приложений** — notepad++, winrar, 7-zip и др.
- ✅ **Шаблоны приложений** — быстрое создание новых экземпляров из шаблонов
- ✅ **Ярлыки рабочего стола** — автоматическое создание .desktop файлов
- ✅ **Создание образов** — резервное копирование WINEPREFIX в архивы

Библиотеки и компоненты

- ✅ **Winetricks библиотеки** — dotnet48, vcrun2013, vcrun2022 и др.
- ✅ **Автоматическое кэширование** — умное управление кэшем winetricks
- ✅ **Проверка зависимостей** — автоматическая установка необходимых компонентов

⚡ Автоматизация системы

Обновление ОС

- ✅ **Полное обновление** — apt update && apt upgrade с автоматическими ответами
- ✅ **Многоуровневая автоматизация** — переменные окружения, опции dpkg, интерактивное распознавание
- ✅ **Умный парсинг** — распознавание этапов (загрузка, распаковка, настройка) с точным прогрессом

Управление репозиториями

- ✅ **Проверка репозиториев** — автоматическая диагностика /etc/apt/sources.list
- ✅ **Активация/деактивация** — управление репозиториями через GUI
- ✅ **Восстановление** — автоматическое исправление проблем конфигурации

Установка компонентов

- ✅ **Astra.IDE** — среда разработки для Astra Linux

- ✅ **APT пакеты** — установка системных пакетов через менеджер
- ✅ **Групповая установка** — выбор нескольких компонентов через чекбоксы
- ✅ **Разрешение зависимостей** — автоматическая установка зависимых компонентов

Офлайн-репозиторий (универсальный архив)

- ✅ **Универсальный архив** — один файл `Update_Astra_XX.tar.gz` с поддержкой нескольких ядер в одной структуре (`universal/common/pool`, `universal/kernel-<ядро>/pool`, `manifest.txt`).
- ✅ **Первая сборка** — на машине с интернетом создаётся структура под текущее ядро; пакеты, специфичные для ядра (linux-image-, linux-headers-, linux-modules-), кладутся в `kernel-<uname -r>/pool`, остальные — в `common/pool`.
- ✅ **Дополнение (та же машина)** — при повторном запуске создания архива добавляются только новые или изменившиеся пакеты (сравнение по имени и размеру); дубликаты в common не создаются; при том же имени, но другом размере вариант сохраняется в kernel-pool.
- ✅ **Другое ядро / другая машина** — скопированный универсальный архив можно дополнять на другой сборке: добавляется `kernel-<текущее ядро>/` и недостающие пакеты, переупаковка в тот же файл.
- ✅ **Использование на целевой машине** — в каталог `AstraPack/Update/` кладётся готовый архив `Update_Astra_XX.tar.gz`. При обновлении ОС из архива извлекаются только `common` и `kernel-<текущее ядро>`, по ним строится репозиторий в `/var/cache/apt/offline-repo` и он подключается к APT.

📊 Мониторинг и диагностика

Процессы

- ✅ **Отслеживание процессов** — мониторинг запущенных процессов Wine и системных служб
- ✅ **Дерево процессов** — визуализация иерархии родительских/дочерних процессов
- ✅ **Использование ресурсов** — CPU, память, сетевая активность

Файловая система

- ✅ **Мониторинг изменений** — отслеживание добавленных/измененных/удаленных файлов
- ✅ **Снимки состояния** — сравнение до/после установки компонентов
- ✅ **Фильтрация** — исключение системных директорий для чистого отчета

Системная информация

- ✅ **Детальная информация** — дистрибутив, ядро, архитектура, оборудование
- ✅ **Мониторинг ресурсов** — дисковое пространство, память, CPU в реальном времени
- ✅ **Wine статус** — версия, путь, WINEPREFIX, активные процессы

🎨 Графический интерфейс

Вкладки приложения

1️⃣ Главная

- Быстрый доступ к основным функциям
- Статус компонентов (Wine, Astra.IDE)
- Системные ресурсы

2️⃣ Обновление ОС

- Автоматическое обновление системы
- Детальные прогресс-бары (загрузка, распаковка, настройка)
- Таблица обновляемых пакетов

3️⃣ Установка программ

- Таблица компонентов с чекбоксами
- Кнопки установки/удаления выбранных
- Создание образов WINEPREFIX
- Статусы установки в реальном времени

4️⃣ Wine

- Управление WINEPREFIX
- Создание/удаление экземпляров приложений
- Настройка Wine окружения
- Установка Windows приложений

5️⃣ Терминал

- Встроенный терминал с логами
- Переключение между потоками (MAIN, ANALYSIS, ERRORS)
- Replay логов для отладки
- Экспорт логов

6️⃣ Мониторинг

- Таблица процессов с деревом
- Фильтрация по имени, PID, CPU
- Мониторинг CPU/NET в реальном времени
- Завершение процессов

7️⃣ Пакеты

- Управление установленными пакетами
- Поиск и фильтрация
- Установка/удаление через APT
- Информация о пакетах

8️⃣ Репозитории

- Список APT репозиториев
- Активация/деактивация источников
- Проверка и восстановление конфигурации

9️⃣ Файловая система

- Мониторинг изменений файлов
- Снимки до/после операций
- Фильтрация по типу изменений
- Детальная информация о файлах

🔟 О системе

- Информация о программе
- Системные характеристики
- Кнопки документации
- Проверка обновлений

💻 Использование

Варианты сборки и запуск (готовые бинарники)

Проект поставляется в виде автономных ELF-файлов (PyInstaller). Существует **3 варианта сборки** (см. `build.json`):

- **full** — полный функционал интерфейса.
- **auto (суффикс `Auto`)** — облегчённый вариант для автономных/упрощённых сценариев (часть вкладок и мониторинговых функций исключена).
- **min (суффикс `Min`)** — минимальный вариант, максимально компактный по возможностям (часть вкладок и мониторинговых функций исключена).

Для каждой поддерживаемой Astra-платформы собирается свой бинарник:

- Astra 1.7: `FSA-AstraInstall-1-7`, `FSA-AstraInstall-Auto-1-7`, `FSA-AstraInstall-Min-1-7`
- Astra 1.8: `FSA-AstraInstall-1-8`, `FSA-AstraInstall-Auto-1-8`, `FSA-AstraInstall-Min-1-8`

Запуск для пользователя:

```
chmod +x ./FSA-AstraInstall-1-7
./FSA-AstraInstall-1-7
```

Вариант `Auto`/`Min` запускается аналогично (просто другой файл).

Важно про переносимость: бинарник можно **копировать в любое место**. На старте приложение само определяет “каноническое” имя сборки и, при необходимости, приводит размещение к рабочей схеме установки (чтобы все каталоги данных/архивов/обновлений были в ожидаемых местах).

Графический режим (GUI)

```
# Простой запуск (автоматически запросит sudo при необходимости)
./FSA-AstraInstall-1-7

# Или для Astra 1.8
./FSA-AstraInstall-1-8

# Для разработки из исходников
python3 Code/Run/FSA-AstraInstall.py
```

Консольный режим (автоматизация)

```
# Полная автоматизация: репозитории + обновление + установка компонентов
sudo ./FSA-AstraInstall-1-7 --console

# Что происходит:
# 1. Проверка и настройка репозиториев
# 2. Обновление системы (apt update && apt upgrade)
# 3. Установка отсутствующих компонентов
# 4. Автоматические ответы на все запросы
```

🔧 Технические требования

Платформы

Платформа	Версия	Статус	GLIBC
Astra Linux	1.7.x (Орел)	✅ Полная поддержка	2.28+
Astra Linux	1.8.x (Орел)	✅ Полная поддержка	2.28+
Ubuntu	20.04+	✅ Полная поддержка	2.31+
Debian	10+ (Buster)	✅ Полная поддержка	2.28+

Системные требования

- **ОЗУ:** минимум 2 ГБ (рекомендуется 4 ГБ+)
- **Диск:** минимум 4 ГБ свободного места
- **Права:** sudo/root для установки системных компонентов
- **Python:** 3.6+ (для запуска из исходников)

📦 Структура бинарников

Автономные исполняемые файлы

Бинарники собраны через **PyInstaller** (запуск сборки из **FSA-DockerManager-Razrab**) и содержат:

- ✅ Python-код приложения и зависимости по `build.json`
- ✅ Все зависимости
- ✅ Tkinter библиотеки
- ✅ Документацию (README.md, HELPME.md)
- ✅ Конфигурационные файлы

Формат: ELF 64-bit LSB executable (типичный размер порядка десятков МБ; точное значение зависит от варианта сборки)

🔄 Система обновлений

Автоматическая проверка

- ✅ **При запуске** — проверка новых версий из настроенных источников
- ✅ **Множественные источники** — SMB сервер, Git репозиторий, HTTPS
- ✅ **Приоритеты источников** — автоматический выбор наиболее доступного
- ✅ **Интерактивное обновление** — подтверждение пользователя перед применением

Источники обновлений

1. **SMB сервер** (приоритет 1) — внутренний корпоративный сервер
2. **Git репозиторий** (приоритет 2) — локальный или удаленный Git
3. **HTTPS** (приоритет 3) — публичные источники

📚 Документация

Файлы документации

- **README.md** — Документация проекта (этот файл)
- **HELPME.md** — Руководство пользователя
- **MEMORANDUM.md** — Репозитории FSA, роли и сборка (связь с DockerManager)
- **CHRONOLOGY.md** — История разработки
- **DocInstruction/** — Техническая документация
  - `ALGORITHM_BEHAVIOR.md` — Алгоритмы поведения
  - `STARTUP_ALGORITHM.md` — Алгоритм запуска
  - `WINE_INSTALL_GUIDE.md` — Руководство по Wine
  - `TABLE_CREATION_GUIDE.md` — Создание таблиц
  - `PYRIGHTCONFIG_DOC.md` — Настройка Pyright

Доступ к документации

В GUI: Вкладка "О системе" → Кнопки "О Программе" / "Справка"

🛠️ Разработка

Запуск из исходников

```
# Клонирование репозитория разработки
git clone <url-FSA-AstraInstall-Razrab>
cd FSA-AstraInstall-Razrab

# Запуск GUI (на машине разработчика; на целевой Astra — см. готовые
бинарники)
python3 Code/Run/FSA-AstraInstall.py

# Запуск консольного режима
sudo python3 Code/Run/FSA-AstraInstall.py --console
```

Сборка бинарников

Сборка выполняется **удалённо** (Docker на сервере) через проект **FSA-DockerManager-Razrab**.

1. Клонировать рядом (или в общий workspace) репозиторий **FSA-DockerManager-Razrab**.
2. В `build.py` и `RunScript/build.py` задан путь к этому клону (по умолчанию путь внутри общего workspace на Mac); при другой раскладке каталогов **исправьте переменную** `docker_manager_path` в этих скриптах.
3. Из **корня FSA-AstraInstall-Razrab**:

```
python3 build.py
# или
python3 RunScript/build.py
```

Скрипт проходит по вариантам и платформам из `build.json` (например astra-1.7 / astra-1.8 и full / auto / light). Готовые ELF-файлы появляются в корне репозитория с именами согласно конфигу (например `FSA-AstraInstall-1-7`, `FSA-AstraInstall-Auto-1-7` и т.д.).

GUI менеджера сборок (DockerManager) находится только в репозитории **FSA-DockerManager-Razrab**. Из корня того клона (см. его `README.md`):

```
cd /path/to/FSA-DockerManager-Razrab
python3 run.py --gui
```

Альтернативно: точка входа `Code/Run/FSA-DockerManager.py` в том же репозитории — детали в документации DockerManager. В **FSA-AstraInstall-Razrab** отдельного запуска GUI сборщика нет

(вложенная папка `DockerManager/` и скрипт `RunScript/docker_run_gui.py` не используются и в репозитории отсутствуют).

🏢 О компании

ООО "НПА Вира-Реалтайм"

- **Сайт:** https://rlt.ru/
- **Email разработчика:** s.fokin@rlt.ru
- **Разработчик:** @FoksSegr

📄 Лицензия

Проприетарное программное обеспечение
© 2025-2026 ООО "НПА Вира-Реалтайм"

🎯 Ключевые преимущества

✨ **Модульная архитектура** — чистый код, легко поддерживать и расширять
⚡ **Быстрый старт** — готовые бинарники, запуск одной командой
🎨 **Современный интерфейс** — интуитивный GUI с вкладками
🤖 **Полная автоматизация** — консольный режим для серверов
📊 **Мониторинг всего** — процессы, ресурсы, файловая система
🔄 **Автообновления** — всегда актуальная версия
🛡️ **Стабильность** — протестировано на Astra Linux 1.7/1.8
📚 **Документация** — подробные руководства и справка

FSA-AstraInstall — профессиональное решение для автоматизации Linux-систем! 🚀